

Project: Nepal Earthquake Mainshock Identification and ShakeMap Animation

Objective

To separate mainshocks from aftershocks in Nepal's earthquake catalog (magnitude ≥ 5) using the Gardner-Knopoff method, then estimate ground shaking (PGA) for each mainshock and produce a cinematic animation of the seismic hazard evolution (1978–2025).

Data Sources

- USGS earthquake catalog (1978–2025, $M \geq 5$, Nepal region)
- Nepal boundary shapefile
- USGS tectonic plates layer (for map background)

Process

1. Filter earthquakes to those inside Nepal.
2. Sort events by time (oldest first).
3. Apply Gardner-Knopoff declustering:
 - First event is a mainshock.
 - For each later event, check if it falls within the time and distance window of any previous mainshock.
 - If yes → aftershock (remove).
 - If no → new mainshock (keep).
4. Save the final mainshock list as a CSV file.

Gardner-Knopoff Windows (used in the code)

Magnitude	Time (days)	Distance (km)
5.0–5.4	15	40

Magnitude	Time (days)	Distance (km)
5.5–6.4	30	50
6.5–7.4	60	75
≥7.5	180	200 (enlarged for Nepal)

Workflow

Load CSV → Extract lat/lon → Filter to Nepal → Sort by time → Apply windows → Save mainshocks. → ApplyPGA → Create ShakeMaps → Animation.mp4

Results

- Original events inside Nepal: **96**
- Mainshocks after declustering: **~58**

PGA Estimation (Boore & Atkinson, 2008)

For each mainshock, a grid of points around the epicentre was created. The hypocentral distance $R_{hypo} = \sqrt{R_{epi}^2 + h^2}$ (h = depth) was computed, and the Boore & Atkinson (2008) GMPE was applied to estimate PGA (in %g) at each grid cell. Coefficients from the paper were used, assuming a rock site ($V_{s30} = 760$ m/s). The resulting PGA grids were stored and later used to draw contour heatmaps in the animation.

Animation (Python + Cartopy)

A cinematic 1920×1080, 60 fps MP4 animation was created with the following features:

- **Full Nepal map:** yellow circles sized by magnitude, with a histogram inset (magnitude distribution) for the first 2 seconds.
- **Zoom to each mainshock** (chronological order):
 - Red star at epicentre, label with date, magnitude, depth, and epicentral PGA.

- 10-level filled contour of PGA (hot colormap, white contour lines).
 - North arrow (top-right), manual scale bar (50 km, bottom-right), data source and CRS text.
- **Smooth camera interpolation** between keyframes (linear zoom, 10 intermediate frames per transition).
- **Closing message** (dark red box, gold border, drop shadow) about disaster risk reduction.
- **Export:** `matplotlib.animation` with `FFMpegWriter`

Animation Code (Python)

The full animation script (about 300 lines) is available in the project repository. It includes:

- Pre-computation of PGA grids for all events
- Keyframe definition and interpolation
- Cartopy map rendering with land/ocean, coastlines, Nepal boundary
- Dynamic label and contour generation per frame

Output

- **Video file:** `NepalEarthquakes.mp4` (1080p)
- **Mainshock catalog:** `nepal_mainshocks_final.csv` (58 events)

Limitations

- Windows were originally made for California; we enlarged the $M \geq 7.5$ window for Nepal.
- The method assumes aftershocks happen equally in all directions (not always true).
- Boore & Atkinson GMPE uses a point-source approximation; finite-fault geometry would improve near-field accuracy for large events.
- Uniform rock site assumed ($V_{s30} = 760$ m/s); actual site effects would vary.

Conclusion

The Gardner-Knopoff method successfully cleaned the Nepal earthquake catalog, and the Boore & Atkinson GMPE provided realistic PGA estimates. The final cinematic

animation visualises 47 years of seismic hazard in a single, professional video, suitable for disaster risk communication, engineering planning, and academic portfolios.

Code (Python) for Gardner-Knopoff declustering.

```
import pandas as pd, json, shapefile
from shapely.geometry import Polygon, Point
from math import radians, sin, cos, sqrt, atan2

# Load Nepal shape
sf = shapefile.Reader("nepal.shp")
nepal_poly = Polygon(sf.shapes()[0].points)

# Load earthquakes
df = pd.read_csv("earthquakes.csv")
df['lon'], df['lat'] = zip(*df['geo'].apply(lambda x: json.loads(x)['coordinates']))

# Keep points inside Nepal
df = df[df.apply(lambda r: nepal_poly.contains(Point(r['lon'], r['lat'])), axis=1)]

# Convert time and sort
df['time'] = pd.to_datetime(df['time'])
df = df.sort_values('time')

# Windows
def window(mag):
    if mag >= 7.5: return 180, 200
    if mag >= 6.5: return 60, 75
    if mag >= 5.5: return 30, 50
    return 15, 40

# Haversine distance
def dist(lon1,lat1,lon2,lat2):
    R=6371; dlat,dlon = map(radians, [lat2-lat1, lon2-lon1])
    a = sin(dlat/2)**2 + cos(radians(lat1))*cos(radians(lat2))*sin(dlon/2)**2
    return R*2*atan2(sqrt(a), sqrt(1-a))

# Decluster
mains = []
for _, e in df.iterrows():
    is_aft = False
    for m in mains:
        if (e['time']-m['time']).days <= window(m['mag'])[0] and \
            dist(m['lon'],m['lat'],e['lon'],e['lat']) <= window(m['mag'])[1]:
            is_aft = True
            break
    if not is_aft:
        mains.append(e)

# Save
pd.DataFrame(mains).to_csv("mainshocks.csv", index=False)
```

Code (Python) for PGA(Boore and Atkinsons(2008))

```
# =====
# NEPAL EARTHQUAKE SHAKEMAPS – FINAL (NO "END OF SEQUENCE" TEXT)
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import geopandas as gpd
import cartopy.crs as ccrs
import cartopy.feature as cfeature
from matplotlib import colors
import warnings
warnings.filterwarnings("ignore")

# =====
# CONFIGURATION – CHANGE PATHS IF NEEDED
# =====
EARTHQUAKE_CSV = r"C:\Users\Lenovo\Desktop\earthquake\nepal_mainshocks_final.csv"
NEPAL_SHP = r"C:\Users\Lenovo\Downloads\usgs earth explorer\kathmandu valley\nepal.shp"
OUTPUT_MP4 = r"C:\Users\Lenovo\Downloads\nepal_final_no_end_text.mp4"
OUTPUT_GIF = r"C:\Users\Lenovo\Downloads\nepal_final_no_end_text.gif"

PAUSE_FULL_START = 2.0    # 2 seconds at start
PAUSE_FULL_END = 2.0      # 2 seconds at end
PAUSE_EVENT = 1.5
TRANSITION_FRAMES = 10
FPS = 60
GRID_STEP = 0.05
FIG_WIDTH = 19.2
FIG_HEIGHT = 10.8
ASPECT_RATIO = FIG_WIDTH / FIG_HEIGHT

# =====
# BOORE & ATKINSON (2008) GMPE
# =====
def pga_boore_atkinson(magnitude, r_rup_km, vs30=760.0):
    M = magnitude
    R = np.sqrt(r_rup_km**2 + 7.8**2)
    c1, c2, c4, c5 = 0.665, -0.122, -0.639, -0.243
    c3 = 0.0 if M < 6.5 else 0.1
    ln_pga = c1 + c2*(M-6) + c3*(M-6)**2 + c4*np.log(R) + c5*np.log(vs30/1130.0)
    return np.exp(ln_pga) # PGA in g

# =====
# LOAD DATA
# =====
print("Loading data...")
df = pd.read_csv(EARTHQUAKE_CSV)
df['datetime'] = pd.to_datetime(df['datetime'], format='mixed', utc=True)
df['datetime'] = df['datetime'].dt.tz_localize(None)
df = df.sort_values('datetime').reset_index(drop=True)
print(f"Loaded {len(df)} mainshocks.")

nepal = gpd.read_file(NEPAL_SHP)
nepal_bounds = nepal.total_bounds
full_extent = [nepal_bounds[0], nepal_bounds[2], nepal_bounds[1], nepal_bounds[3]]

# =====
```

```

# MAGNITUDE-DEPENDENT ZOOM
# =====
def get_zoom_buffer(mag):
    if mag >= 7.5: return 2.0
    elif mag >= 6.5: return 1.5
    elif mag >= 5.5: return 1.2
    else: return 1.0

# =====
# PRE-COMPUTE PGA GRIDS
# =====
print("Pre-computing PGA grids...")
shakemaps = {}
R_earth = 6371

for idx, row in df.iterrows():
    lon0, lat0, mag = row['longitude'], row['latitude'], row['mag']
    depth = row['depth']
    dt = row['datetime']
    base_buf = get_zoom_buffer(mag)
    buf_lat = base_buf
    buf_lon = base_buf * ASPECT_RATIO
    lon_min = lon0 - buf_lon
    lon_max = lon0 + buf_lon
    lat_min = lat0 - buf_lat
    lat_max = lat0 + buf_lat
    lon_vec = np.arange(lon_min, lon_max, GRID_STEP)
    lat_vec = np.arange(lat_min, lat_max, GRID_STEP)
    lon_grid, lat_grid = np.meshgrid(lon_vec, lat_vec)
    dlon = np.radians(lon_grid - lon0)
    dlat = np.radians(lat_grid - lat0)
    lat0_rad = np.radians(lat0)
    a = np.sin(dlat/2)**2 + np.cos(lat0_rad) * np.cos(np.radians(lat_grid)) * np.sin(dlon/2)**2
    epi_km = 2 * R_earth * np.arctan2(np.sqrt(a), np.sqrt(1-a))
    hypo_km = np.sqrt(epi_km**2 + depth**2)
    pga_grid = np.zeros_like(epi_km)
    for i in range(epi_km.shape[0]):
        for j in range(epi_km.shape[1]):
            pga_g = pga_boore_atkinson(mag, hypo_km[i,j])
            pga_grid[i,j] = pga_g * 100.0
    shakemaps[dt] = {
        'pga': pga_grid,
        'lon': lon_vec,
        'lat': lat_vec,
        'extent': [lon_min, lon_max, lat_min, lat_max]
    }
    if (idx+1) % 10 == 0:
        print(f' Processed {idx+1}/{len(df)}')
print("All grids ready.\n")

# =====
# BUILD KEYFRAMES (no "End of sequence" subtitle)
# =====
keyframes = [{'extent': full_extent, 'point': None,
              'title': 'Nepal Earthquake ShakeMaps',
              'subtitle': ''}] # empty at start – no extra text

for idx, row in df.iterrows():
    lon, lat, mag = row['longitude'], row['latitude'], row['mag']
    depth = row['depth']

```

```

date_str = row['datetime'].strftime('%Y-%m-%d')
place = row['place']
base_buf = get_zoom_buffer(mag)
buf_lat = base_buf
buf_lon = base_buf * ASPECT_RATIO
extent = [lon - buf_lon, lon + buf_lon, lat - buf_lat, lat + buf_lat]

label = (f"Date: {date_str}\n"
        f"Magnitude: {mag:.1f}\n"
        f"Depth: {depth} km\n"
        f"{place[:40]}")
keyframes.append({
    'extent': extent,
    'point': {
        'lon': lon, 'lat': lat, 'mag': mag, 'depth': depth,
        'date': date_str, 'place': place, 'datetime': row['datetime'], 'label': label
    },
    'title': 'Nepal Earthquake ShakeMaps',
    'subtitle': f"{date_str} M{mag:.1f}"
})
# Final keyframe: no subtitle, no point
keyframes.append({'extent': full_extent, 'point': None,
                  'title': 'Nepal Earthquake ShakeMaps',
                  'subtitle': ""})

# =====
# FRAME INTERPOLATION
# =====
def interpolate_extent(e1, e2, t):
    return [e1[0] + t*(e2[0]-e1[0]),
            e1[1] + t*(e2[1]-e1[1]),
            e1[2] + t*(e2[2]-e1[2]),
            e1[3] + t*(e2[3]-e1[3])]

frames = []
for i in range(len(keyframes)-1):
    start, end = keyframes[i], keyframes[i+1]
    for step in range(TRANSITION_FRAMES):
        t = step / TRANSITION_FRAMES
        extent = interpolate_extent(start['extent'], end['extent'], t)
        point = end['point'] if t >= 0.9 else None
        frames.append({
            'extent': extent,
            'point': point,
            'title': end['title'] if t >= 0.9 else start['title'],
            'subtitle': end['subtitle'] if t >= 0.9 else start['subtitle']
        })
    frames.append({
        'extent': end['extent'],
        'point': end['point'],
        'title': end['title'],
        'subtitle': end['subtitle']
    })
if end['point'] is not None:
    for _ in range(int(PAUSE_EVENT * FPS) - 1):
        frames.append({
            'extent': end['extent'],
            'point': end['point'],
            'title': end['title'],
            'subtitle': end['subtitle']
        })

```



```

    })

first = frames[0]
for _ in range(int(PAUSE_FULL_START * FPS) - 1):
    frames.insert(0, first)
last = frames[-1]
for _ in range(int(PAUSE_FULL_END * FPS)):
    frames.append(last)
print(f"Total frames: {len(frames)}")

# =====
# DRAWING FUNCTION – NO "END OF SEQUENCE" TEXT
# =====
fig = plt.figure(figsize=(FIG_WIDTH, FIG_HEIGHT), dpi=100)
fig.patch.set_facecolor('black')
ax = plt.axes(projection=ccrs.PlateCarree())
ax.set_facecolor('black')

# PGA colour bar
cmap = plt.cm.hot
norm = colors.Normalize(vmin=0, vmax=100)
cbar_ax = fig.add_axes([0.15, 0.02, 0.7, 0.02])
cbar = fig.colorbar(plt.cm.ScalarMappable(norm=norm, cmap=cmap),
                    cax=cbar_ax, orientation='horizontal')
cbar.set_label('Peak Ground Acceleration (%g)', fontsize=12, color='white')
cbar.ax.tick_params(colors='white')
cbar.ax.xaxis.label.set_color('white')

def circle_size(mag):
    return 24 + (mag - 5) * 36

def draw(frame):
    ax.clear()
    extent = frame['extent']
    point = frame['point']
    title = frame['title']
    subtitle = frame['subtitle']

    ax.set_extent(extent, crs=ccrs.PlateCarree())
    ax.set_facecolor('black')

    # Background
    ax.add_feature(cfeature.OCEAN, facecolor='lightblue', zorder=0)
    ax.add_feature(cfeature.LAND, facecolor='lightgray', zorder=0)
    ax.add_feature(cfeature.COASTLINE, linewidth=0.5, edgecolor='white', zorder=1)
    nepal.boundary.plot(ax=ax, edgecolor='blue', linewidth=2, facecolor='none', zorder=2)

    # Main content
    if point is None and extent == full_extent:
        sizes = [circle_size(m) for m in df['mag']]
        ax.scatter(df['longitude'], df['latitude'], c='gold', s=sizes,
                  alpha=0.9, edgecolors='darkgoldenrod', linewidth=0.8,
                  transform=ccrs.PlateCarree(), zorder=3)
    elif point is not None:
        dt = point['datetime']
        sm = shakemaps.get(dt)
        if sm is not None:
            X, Y = np.meshgrid(sm['lon'], sm['lat'])
            ax.contourf(X, Y, sm['pga'], levels=10, cmap=cmap, alpha=0.6,
                      transform=ccrs.PlateCarree(), zorder=1)

```

```

ax.contour(X, Y, sm['pga'], levels=5, colors='white', linewidths=0.5,
           alpha=0.7, transform=ccrs.PlateCarree(), zorder=1)
ax.plot(point['lon'], point['lat'], 'r*', markersize=16,
        markedgecolor='black', transform=ccrs.PlateCarree(), zorder=5)

if sm is not None:
    lon_idx = np.argmin(np.abs(sm['lon'] - point['lon']))
    lat_idx = np.argmin(np.abs(sm['lat'] - point['lat']))
    pga_val = sm['pga'][lat_idx, lon_idx]
    label_text = point['label'] + f"\nPGA at epicentre: {pga_val:.1f}%g"
else:
    label_text = point['label']

ax.annotate(label_text, xy=(point['lon'], point['lat']), xytext=(15, 15),
            textcoords='offset points', fontsize=12, color='white',
            bbox=dict(boxstyle="round,pad=0.3", facecolor='black', alpha=0.9, edgecolor='white'),
            arrowprops=dict(arrowstyle='->', color='white'),
            transform=ccrs.PlateCarree(), zorder=5)

# Scale bar (bottom-right)
if (extent[1] - extent[0]) < 5.0:
    xmin, xmax = extent[0], extent[1]
    ymin, ymax = extent[2], extent[3]
    mid_lat = (ymin + ymax) / 2
    km_per_deg = 111.0 * np.cos(np.radians(mid_lat))
    bar_len_deg = 50.0 / km_per_deg
    if bar_len_deg < (xmax - xmin) * 0.8:
        bar_start = xmax - bar_len_deg - 0.1
        bar_end = xmax - 0.1
        bar_y = ymin + 0.05 * (ymax - ymin)
        ax.plot([bar_start, bar_end], [bar_y, bar_y], color='white', linewidth=3, transform=ccrs.PlateCarree())
        ax.text((bar_start + bar_end)/2, bar_y - 0.03, "50 km",
                transform=ccrs.PlateCarree(), color='white', ha='center', fontsize=10)

# North arrow
ax.annotate('N', xy=(0.93, 0.95), xycoords='axes fraction',
            fontsize=18, fontweight='bold', ha='center', va='center',
            color='white', bbox=dict(boxstyle='circle', facecolor='none', edgecolor='white', alpha=0.8))
ax.annotate('↑', xy=(0.93, 0.91), xycoords='axes fraction',
            fontsize=22, ha='center', va='center', color='white')

# Data source and CRS
ax.text(0.02, 0.06, "Data: USGS (M≥5, 1978-2025) | PGA: Boore & Atkinson (2008) | Boundary: Nepal",
        transform=ax.transAxes, fontsize=9, color='white',
        bbox=dict(boxstyle='round', facecolor='black', alpha=0.7, edgecolor='white'))
ax.text(0.02, 0.01, "CRS: EPSG:4326 (WGS84)", transform=ax.transAxes,
        fontsize=9, color='white', ha='left', va='bottom',
        bbox=dict(boxstyle='round', facecolor='black', alpha=0.7, edgecolor='white'))

# Title and subtitle (no subtitle in final frame)
ax.set_title(title, fontsize=28, fontweight='bold', pad=20, color='white')
if subtitle:
    ax.text(0.5, -0.05, subtitle, transform=ax.transAxes, fontsize=16,
           ha='center', va='top', color='white',
           bbox=dict(boxstyle='round', facecolor='black', alpha=0.8, edgecolor='white'))
ax.grid(True, linestyle=':', alpha=0.3, color='white')

# STYLISH CLOSING MESSAGE (only at the end)
if point is None and extent == full_extent and subtitle == "":
    # This will appear only on the final full view (the very last frames)

```

```

# We'll show it only on the last, say, 1 second of the pause.
# For simplicity, we'll just draw it for all final full frames; it's fine.
message_lines = [
    "Earthquakes cannot be prevented,",
    "but their impact can be reduced.",
    "Nepal continues to build resilience",
    "through early warning systems,",
    "building codes, and public awareness.",
    "",
    "Map by: Abhishek Karki"
]
message = "\n".join(message_lines)
ax.text(0.5, 0.49, message, transform=ax.transAxes,
        fontsize=22, color='black', ha='center', va='center',
        fontweight='bold', alpha=0.8)
ax.text(0.5, 0.5, message, transform=ax.transAxes,
        fontsize=22, color='white', ha='center', va='center',
        fontweight='bold',
        bbox=dict(boxstyle="round,pad=0.8", facecolor="#8B0000", edgecolor='gold', linewidth=3, alpha=0.95),
        linespacing=1.8)

return ax

# =====
# CREATE AND SAVE ANIMATION
# =====
ani = animation.FuncAnimation(fig, draw, frames=frames, interval=1000/FPS, repeat=False)

try:
    writer = animation.FFMpegWriter(fps=FPS, codec='h264', bitrate=5000)
    ani.save(OUTPUT_MP4, writer=writer)
    print(f"✅ Animation saved to: {OUTPUT_MP4}")
except Exception as e:
    print(f"MP4 failed ({e}). Saving as GIF...")
    ani.save(OUTPUT_GIF, writer='pillow', fps=FPS//2)
    print(f"✅ GIF saved to: {OUTPUT_GIF}")

print("Animation complete. No 'End of sequence' text; start/end pauses = 2 seconds.")

```